

Two hands, one policy: a controlled study of action-distribution factorisation for bimanual Diffusion Policy

Pedram Aminharati

companion study to Chi et al., *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion* (RSS 2023, IJRR 2024)
September 2026

Abstract

Push-T is a one-finger task. The IJRR version of Diffusion Policy extends the method to three real bimanual tasks and makes one design decision along the way: concatenate both arms into a single action vector and let one diffusion model denoise it. That decision is never ablated, and the bimanual literature disagrees about it — decentralised and leader–follower factorisations are both advocated. We build a two-hand version of Push-T with a scripted expert that uses *force couples*, a manoeuvre a single finger cannot perform, and we train three factorisations of the two-hand action distribution — joint $p(a_1, a_2 | o)$, independent $p(a_1 | o)p(a_2 | o)$ and leader–follower $p(a_1 | o)p(a_2 | o, a_1)$ — with the paper’s own recipe on *identical* demonstrations, so that the only variable is the factorisation. On 50 unseen initial states, joint (0.964 mean coverage) and leader–follower (0.963) are indistinguishable and both beat independent sampling (0.941, -12 points of success), and the two winners never fail catastrophically (min coverage 0.94 and 0.93 over 50 episodes) whereas the scripted teacher does. We also report the sampling-cost curve of conditional flow matching against DDPM/DDIM (a $100\times$ latency reduction for four points of coverage) and a negative result we consider the most useful part of the study: three rounds of DAgger with a queryable expert *degrade* the policy monotonically, and we trace the cause by measuring what is in the labels rather than by tuning.

1 Introduction

Diffusion Policy [1] represents a visuomotor policy as a conditional denoising process over an *action sequence*. Its bimanual experiments — egg beater, mat unrolling, shirt folding — extend the proprioceptive and image observation spaces to both arms and the action space to the desired poses and gripper widths of both end effectors, and report that the method “worked out of the box . . . without hyperparameter tuning”. The two arms therefore appear in exactly one place: as a longer action vector that a single denoiser produces jointly.

That is a modelling choice, not a neutral implementation detail. Written as a probability, it says that the policy samples a_1 and a_2 from one joint distribution conditioned on the observation, so a sampled plan for one hand is automatically consistent with the plan for the other. The bimanual literature contains at least two alternatives. Decentralised policies [6]

train per-agent diffusion policies with a centralised objective and execute them from local observations. Role decompositions [5] split the task into a stabilising arm and an acting arm, i.e. a leader–follower factorisation, and report large gains over an unstructured joint baseline on real tasks. In-context bimanual policies [7] compare precisely these three families and find leader–follower best — but with a frozen language model doing the regression, so the factorisation is confounded with the model class. In multi-agent reinforcement learning the same three options are long-standing [16, 17], as they are in trajectory forecasting [18].

Nobody, as far as we can find, has held the model, the recipe and the data fixed and changed only the factorisation. That is what this study does, on a task small enough that every variable is controlled and every rollout can be watched.

Contributions. (i) A two-hand extension of Push-T with a scripted force-couple expert; it is cheap, deterministic, reproduces the original seeds, and — unlike a human demonstrator — can be queried at any state, which makes DAgger experiments possible. (ii) A controlled comparison of three factorisations under one training recipe and one demonstration set, with coordination metrics as well as task metrics. (iii) The interaction of that choice with the training objective (DDPM versus conditional flow matching), with on-policy data aggregation, and with ACT-style temporal ensembling. (iv) A documented negative result: naive DAgger degrades the policy here, and the measured cause is the composition of the labels, not the algorithm.

2 Related work

Factorising two arms. The joint vector is the default at scale: RDT-1B [13] and π_0 [14] denoise both arms in one action chunk, and 3D FlowMatch Actor [15] states the rationale explicitly. Architectural work keeps the joint output but adds cross-arm structure: InterACT [9] inserts synchronisation blocks between per-arm decoders, IACE [10] an inter-arm encoder. Compositional work goes the other way: AnyBimanual [11] and TwinVLA [12] build a bimanual policy out of single-arm ones, and CoDi [8] couples independently trained single-arm diffusion policies at test time through a cooperative game. Our three arms of comparison are the probabilistic skeletons of these families.

Push-T and planar pushing. Push-T comes from Implicit Behavioral Cloning [2] and is the task on which Diffusion Policy is most often illustrated. We found no published two-hand or multi-agent Push-T. The nearest bimanual planar problem is GLIDE [19], where demonstrations for bimanual $SE(2)$ reorientation are synthesised by contact-implicit trajectory optimisation and a joint diffusion policy is trained on them — a model-based teacher for a learned student, as here.

Coordination metrics. BiCoord [20] proposes simultaneity and relative-distance measures for bimanual policies; MIMIC-D [6] and CoDi [8] count collisions. Our two metrics (hand overlap, both hands near the block) are in that family and are chosen so that a policy which quietly reverts to using one hand is visible in the numbers.

Deployment-side methods. Temporal ensembling comes from ACT [3]; conditional flow matching for policies from [21, 22]; DAGger from Ross et al. [4], with Diff-DAGger [23] triggering the expert only where the diffusion policy is uncertain. Equivariant Diffusion Policy [24] learns the $SO(2)$ structure that we hand-code as a block-frame observation.

3 Testbed

Environment. `PushTTwoHandsEnv` adds a second kinematic disc to the authors’ `PushTEnv`. Physics, goal pose and reward (coverage of the goal region by the T) are untouched; the action is the pair of target positions of both discs, tracked by the same PD controller, so the 4-D action space is the exact analogue of the paper’s “concatenate both arms” design. Seeds reproduce the original random draws, so seed 100000 here is the block pose of seed 100000 there. The state is $[\text{hand}_1, \text{hand}_2, \text{block } xy, \text{block } \theta]$.

Expert. There are no human two-hand demonstrations for Push-T, so demonstrations come from a scripted feedback controller (Fig. 1) built on three grasps expressed in the block frame: PUSH (both discs behind the bar: translation along the stem), CCW/CW couples (one disc on the stem’s flank, one on the bar’s end, pushing in opposite directions with *equal lever arms* about the centroid: rotation in place), and single-disc nudges whose line of action passes through the centroid (pure translation for the last centimetres).

The couple is the reason two hands are necessary. Two contacts at r_1, r_2 applying opposite forces f and $-f$ produce a resultant force and a moment about the centroid c

$$\begin{aligned} F &= f + (-f) = 0, \\ \tau &= (r_1 - c) \times f + (r_2 - c) \times (-f) = (r_1 - r_2) \times f, \end{aligned} \quad (1)$$

so the block turns without translating, and the moment depends only on the *separation* of the contacts. A single disc can only apply a force through one point: it produces $\tau \neq 0$ but also $F \neq 0$, i.e. it drags the block while turning it. Equation (1)

also dictates the geometry — the contact points are placed at equal lever arms (40 px either side of the centroid), because with unequal arms the contact on the shorter arm separates first and a residual F appears; this was verified numerically before the expert was written. The block is quasi-static, so each contact step moves it predictably. Hands travel to a new grasp on arcs around the block, clipped to the arena and checked for collision with the T; grasps whose contacts or staging points are unreachable are declared infeasible and the expert falls back to pushes that move the block off the wall. Its remaining failures ($\approx 3\%$ of random starts) are blocks lying flat against a wall, where no two-disc grasp exists.

Data. 1000 attempts on seeds 0–999; the 968 successful episodes are kept (303,398 steps), as the human Push-T set also keeps only completed episodes. Windows of $T_o = 2$ observations and $T_p = 16$ actions give 296,622 training windows.

4 Method

Notation. At control step t the policy sees the last $T_o = 2$ observations O_t and outputs an action sequence $A_t = (a_t, \dots, a_{t+T_p-1})$ with $T_p = 16$, of which the first $T_a = 8$ are executed. Each action is the pair of disc targets $a = (a^1, a^2) \in \mathbb{R}^4$; superscripts index the hand.

Representation. Let p_b, θ_b be the block pose and $R(\cdot)$ a planar rotation. Observations are expressed in the block frame and actions as *offsets* from the current hand positions in that frame,

$$\tilde{x} = R(-\theta_b)(x - p_b), \quad \bar{a}_{t+j}^i = R(-\theta_b)(a_{t+j}^i - h_t^i), \quad (2)$$

which makes the policy invariant to the block’s pose — the hand-coded version of the $SO(2)$ equivariance that [24] learns — and keeps the regression targets small. Each dimension of $\bar{a}$ is min–max normalised to $[-1, 1]$ using the 99.7th percentile of $|\bar{a}|$, as the paper’s appendix prescribes for a DDPM that clips to that range.

Training objective. All diffusion policies use the paper’s `ConditionalUnet1D` with FiLM conditioning (down-dimensions [128, 256, 512], 256-d step embedding, 17.8 M parameters) and its ε -prediction loss: with the square-cosine schedule $\bar{\alpha}_k$ of iDDPM and $K = 100$ iterations,

$$\mathcal{L}(\theta) = \mathbb{E}_{(O, A^0) \sim \mathcal{D}, k, \varepsilon} \left\| \varepsilon - \varepsilon_\theta(O, \sqrt{\bar{\alpha}_k} A^0 + \sqrt{1 - \bar{\alpha}_k} \varepsilon, k) \right\|^2, \quad (3)$$

$k \sim \mathcal{U}\{1..K\}$, $\varepsilon \sim \mathcal{N}(0, I)$. The network is a scaled score estimator,

$$\nabla_A \log q(A^k | O) = - \frac{\varepsilon_\theta(O, A^k, k)}{\sqrt{1 - \bar{\alpha}_k}}, \quad (4)$$

which is why the intractable normaliser of an energy-based policy never appears: it does not depend on A , so it vanishes

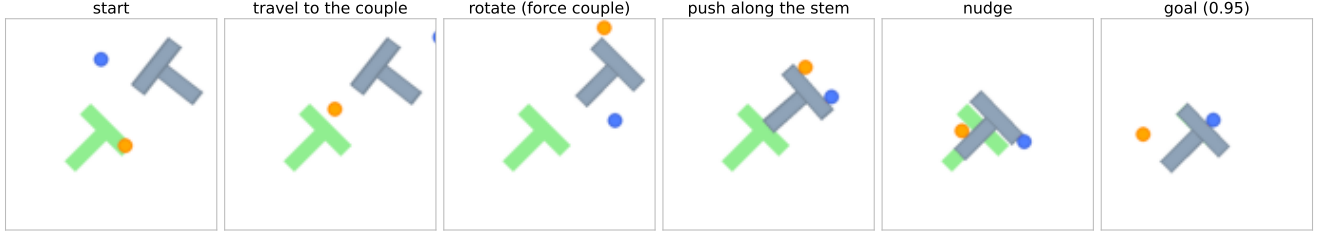


Figure 1: The scripted two-hand expert on evaluation seed 100000 (blue and orange discs, grey T, green goal). It selects a *grasp* in the block frame, travels there on collision-checked arcs, and applies it: two contacts with equal lever arms about the centroid form a force couple and rotate the T in place — the manoeuvre a single finger cannot perform — then both discs push along the stem and single-disc nudges finish the alignment. On 50 unseen initial states the expert reaches 0.945 mean coverage and succeeds on 49 of 50.

under ∇_A . Training uses EMA weights, batch 256, learning rate 10^{-4} , 100 epochs.

The three factorisations. They differ only in how $p_\theta(A^1, A^2 | O)$ is written, and therefore in what the sampler produces:

$$\text{joint: } p_\theta(A^1, A^2 | O) \quad (5)$$

$$\text{independent: } p_{\theta_1}(A^1 | O) p_{\theta_2}(A^2 | O) \quad (6)$$

$$\text{leader-follower: } p_{\theta_1}(A^1 | O) p_{\theta_2}(A^2 | O, A^1) \quad (7)$$

The joint model denoises the 16×4 tensor in one network. The independent model runs two networks with the same loss (3) on the 16×2 half that belongs to each hand — both see the *full* state, so the difference is not observability but the independence of the two noise draws at sampling time. The leader-follower model adds hand 1’s plan to hand 2’s conditioning, $\varepsilon_{\theta_2}(O, \bar{A}^1, A^{2,k}, k)$, teacher-forced with the demonstrated $\bar{A}^1$ during training and with hand 1’s sampled plan at deployment; it costs two sequential network evaluations per decision.

Flow matching. A fourth network is trained on the same windows with conditional flow matching. With flow time $\tau \in [0, 1]$, noise $A^{(1)} \sim \mathcal{N}(0, I)$ and the straight path $A^{(\tau)} = (1 - \tau)A^{(1)} + \tau A^0$,

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{\tau, A^0, A^{(1)}} \|v_\theta(O, A^{(\tau)}, \tau) - (A^0 - A^{(1)})\|^2, \quad (8)$$

and sampling integrates $\dot{A} = v_\theta$ with n Euler steps of size $1/n$; we report $n \in \{1, 4, 10\}$. The network, the data and the conditioning are identical to (3), so the comparison isolates the objective and the sampler.

Deployment. Receding horizon with $T_a = 8$: predict 16 actions, execute 8, re-plan. Samplers: DDPM-100, DDIM-10, and DDIM-10 with ACT-style temporal ensembling (a new plan every step; the executed action is an exponentially weighted average over the plans that cover it).

Metrics. Mean coverage of the goal region (the paper’s metric) and success (coverage > 0.95 , the environment’s own

threshold) on the 50 unseen seeds 100000–100049; the *minimum* coverage over those episodes as a worst-case measure; steps to termination; milliseconds per decision on an A100; and two coordination measures, with R the disc radius, c_t the block centroid and $d = 110$ px,

$$\begin{aligned} \text{overlap} &= \frac{1}{T} \sum_t \mathbf{1}[\|h_t^1 - h_t^2\| < 2R], \\ \text{near} &= \frac{1}{T} \sum_t \mathbf{1}[\|h_t^1 - c_t\| < d \wedge \|h_t^2 - c_t\| < d]. \end{aligned} \quad (9)$$

$$(10)$$

The discs are kinematic and pass through each other, so (9) counts the times the policy sent both hands to the same place — a coordination failure that costs nothing in this simulator but would be a collision on hardware. Equation (10) detects the opposite failure: a policy that quietly reverts to using one hand.

5 Results

Table 1 and Fig. 2 give the main comparison.

The factorisation matters, and the paper’s choice is sound.

Joint (0.964) and leader-follower (0.963) are indistinguishable in mean coverage and in success (0.94 and 0.96; see the variance note below). Independent sampling is clearly worse: 0.941 coverage and 0.84 success, and it is the slowest of the three (409 steps). The failure is exactly the one the multi-agent literature predicts — two individually plausible plans need not be a plausible *pair*. Conditioning hand 2 on hand 1’s sampled plan recovers what independent sampling loses, at the price of two sequential network evaluations (176 against 91 ms). On this task the joint vector therefore buys the same coordination for half the latency, which is a concrete argument for the design the paper adopted without ablating it.

The learned policies are more robust than their teacher.

The expert has the best success rate (0.98) and is the fastest (315 steps), but its worst episode is a complete failure — the wall-jammed blind spot it has by construction. Joint DDIM-10, leader-follower and flow matching Euler-10 never drop below 0.92 coverage on any of the 50 seeds. Behaviour cloning from a thousand demonstrations has smoothed a failure mode of the

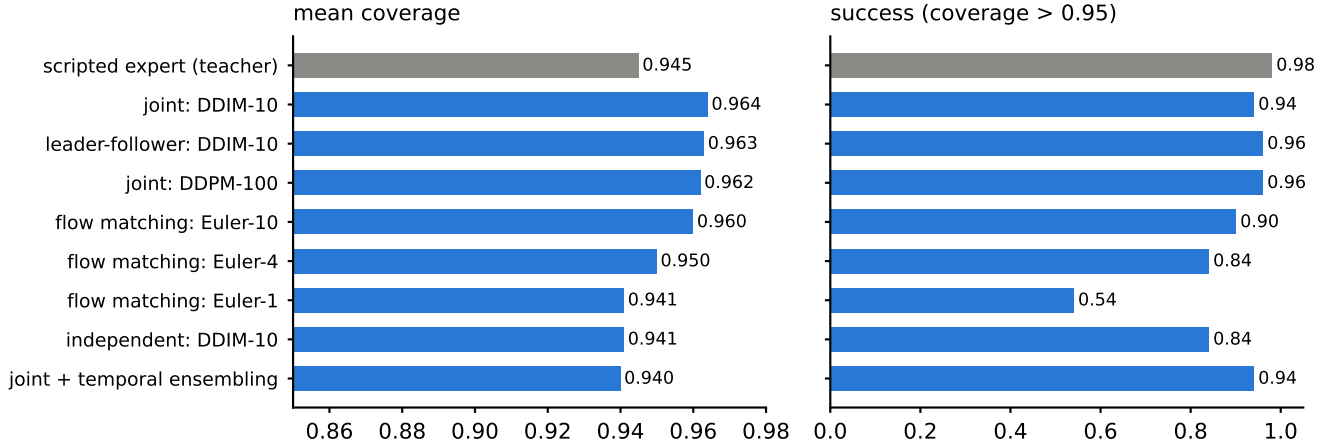


Figure 2: Mean coverage and success on 50 unseen initial states. The joint and leader–follower factorisations are indistinguishable and both beat independent sampling; the scripted teacher (grey) has the highest success rate but not the highest mean coverage, and it is the only method here that fails an episode completely.

Table 1: All variants on the 50 unseen seeds ($T_a = 8$, step budget 800). “min” is the worst episode of the 50; “near” is the fraction of steps with both hands close to the block. Latency is per decision on an A100.

variant	coverage	success	min	steps	near	ms
scripted two-hand expert (teacher)	0.945	0.98	0.000	315	—	—
joint: DDPM-100	0.962	0.96	0.924	379	0.725	894
joint: DDIM-10	0.964	0.94	0.942	397	0.734	91
leader–follower: DDIM-10	0.963	0.96	0.928	360	0.740	176
independent: DDIM-10	0.941	0.84	0.000	409	0.725	176
joint: DDIM-10 + temporal ensembling	0.940	0.94	0.000	382	0.720	87
flow matching: Euler-10	0.960	0.90	0.926	394	0.744	82
flow matching: Euler-4	0.950	0.84	0.499	435	0.741	34
flow matching: Euler-1	0.941	0.54	0.844	551	0.760	8.9
joint + DAgger round 1	0.926	0.52	0.764	574	0.701	87
joint + DAgger round 2	0.805	0.48	0.000	611	0.607	87
joint + DAgger round 3	0.765	0.42	0.000	627	0.599	87

demonstrator: the policy is a little less often above the strict threshold, and much less often catastrophic.

Sampling cost. DDPM-100 and DDIM-10 reach the same coverage at a tenth of the cost (894 $\rightarrow$ 91 ms), reproducing the paper’s own reason for using DDIM on hardware. Conditional flow matching extends the curve: 10 Euler steps match DDIM, 4 steps cost four points of coverage, and a *single* step still reaches 0.941 coverage at 8.9 ms — a hundredfold reduction from DDPM — although success collapses to 0.54, i.e. the one-step policy gets close to the goal and rarely crosses the strict threshold.

Temporal ensembling does not help here. Averaging overlapping plans costs two points of coverage and turns the worst episode into a failure. This is consistent with recent evidence [25] that exponentially weighted ensembling blurs multimodal action distributions; on a task whose whole point is that two grasps are equally valid, averaging across plans is the wrong

operation.

Variance. The study was run twice end to end with different network initialisations. Mean coverage is stable to about ± 0.02 ; success, being a threshold applied to 50 episodes, moved by up to 0.28 on one variant. All conclusions above hold in both runs; we therefore quote coverage as the headline metric and treat success gaps below ten points as noise. Reporting three or more seeds with confidence intervals is the first thing we would add.

6 A negative result: DAgger with a queryable expert

A scripted expert can be queried at any state, which is exactly what human teleoperation cannot provide and what DAgger [4] needs. Writing π^* for the expert and $\hat{\pi}_i$ for the policy after round i , the algorithm rolls out a mixture $\pi_i = \beta_i \pi^* + (1 -$

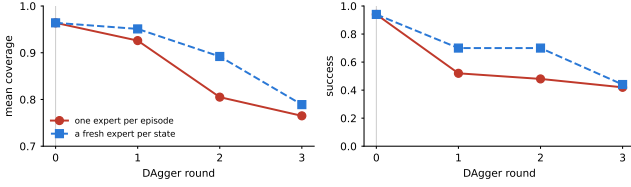


Figure 3: Three rounds of DAGger on the joint policy, with two labelling schemes. Round 0 is the base policy. Both curves fall. Labelling every state with a fresh expert (blue) helps at every round but does not stop the degradation.

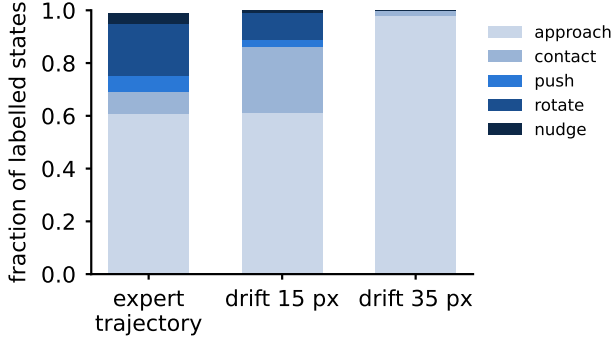


Figure 4: What is *in* the labels. Phase of the scripted expert at the states it is asked to label, on its own trajectory and on rollouts perturbed by 15 and 35 px. As the rollout drifts, the phases that actually move the block — push, rotate, nudge — disappear and the labels become almost entirely “travel back to a grasp”.

$\beta_i) \hat{\pi}_i$, labels every visited state with the expert and aggregates,

$$\mathcal{D}_{i+1} = \mathcal{D}_i \cup \{(o, \pi^*(o)) : o \sim d_{\pi_i}\}, \quad (11)$$

where d_{π_i} is the state distribution induced by π_i . Its guarantee replaces the $O(T^2\epsilon)$ compounding error of behaviour cloning by $O(T\epsilon)$. We ran the textbook version with $\beta_i = 0$ (pure policy rollouts): 100 fresh seeds per round, the expert labelling every visited state, aggregation with the demonstrations, fine-tuning for 20 epochs at 5×10^{-5} , three rounds.

The policy got monotonically worse (Fig. 3): mean coverage $0.964 \rightarrow 0.926 \rightarrow 0.805 \rightarrow 0.765$ and success $0.94 \rightarrow 0.52 \rightarrow 0.48 \rightarrow 0.42$; on the aggregation rollouts themselves, success fell $0.87 \rightarrow 0.62 \rightarrow 0.43$. We tested three explanations.

(1) Normalisation. The first implementation recomputed the min-max and percentile statistics on the aggregated data, whose action offsets are wider because the expert labels states the demonstrations never visit. Every input and output of the pretrained network was therefore silently rescaled. Keeping the base checkpoint’s statistics is correct and improved the second-round rollout success from 0.52 to 0.62 — but the collapse remained.

(2) The expert is not a labelling function. DAGger assumes the expert is a function of the state. Ours is a feedback con-

troller with memory: the grasp it has committed to, the direction its hands are travelling round the block, the grasps it has temporarily blacklisted. On its own trajectory this is invisible — a fresh expert asked the same question answers identically at 96% of states — but on rollouts perturbed by 20 px a fresh expert picks a *different grasp* at 21% of states and an action more than 20 px away at 33%. Labelling every state with a fresh expert is a strictly better-posed procedure and it helps at every round (coverage $+0.025, +0.087, +0.024$; success $+0.18, +0.22, +0.02$), but the policy still degrades. The hypothesis is a real contributing factor and not the dominant cause.

(3) The labels are dominated by recovery. Measuring the expert’s *phase* at the states it labels settles it (Fig. 4). On its own trajectory the distribution is 61% approach, 20% rotate, 6% push. On a rollout that has drifted 35 px it is 98% approach and essentially 0% of the phases that move the block. Aggregating 40–60k such states with 300k demonstration steps shifts the training mixture towards travelling. The symptoms match precisely: the fine-tuned policies take far more steps ($397 \rightarrow 574 \rightarrow 611 \rightarrow 627$) and keep their hands away from the block (both-near $0.72 \rightarrow 0.60$) until they exhaust the step budget. The policy did not forget how to push; it learned to reposition.

What follows. The remedy is the β_i we had set to zero in (11). With $\beta_i > 0$ the rollout is a mixture, so the visited states — and therefore the label composition of Fig. 4 — stay near the demonstration distribution; Ross et al. require only $\beta_i \rightarrow 0$ for the guarantee to hold, which the usual schedule $\beta_i = p^{i-1}$ satisfies. Diff-DAGger’s uncertainty gating [23] addresses the same problem from the other side by asking the expert only where the policy is unsure. We regard the episode as the most transferable part of this study: on a real line the failure would have looked like a policy that mysteriously slows down, and the diagnosis came from measuring the composition of the training data, not from tuning the optimiser.

7 Limitations

Two discs on a plane are not two seven-degree-of-freedom arms: there is no reach, no grasp, no force control, no perception. What transfers is the statistical question — how to write the joint distribution of two actuators’ actions — and the deployment questions around it. The demonstrations come from a scripted controller, so every policy inherits its style and its wall-jammed blind spot; results with human demonstrations, which are more multimodal, could differ, and multimodality is precisely where factorisation should matter most. The comparison uses two training seeds; three or more with confidence intervals are needed for the success metric. Finally, the study tests one task. The natural next steps are a second task with tighter coupling (two discs carrying a bar, or a hand-over), a data-scaling sweep to find where independent sampling breaks,

and the same three factorisations on an established two-arm benchmark such as robomimic Transport, which Diffusion Policy already reports.

8 Conclusion

Holding the model, the recipe and the demonstrations fixed, the joint action vector that Diffusion Policy adopts for its bimanual tasks is a sound default: it matches an explicit leader–follower factorisation and clearly beats independent per-hand sampling, at half the latency. The learned policies end up more robust than the scripted teacher they imitate. And the most useful outcome was a failure: DAgger, applied by the book with an expert that can be queried anywhere, made the policy worse, and the cause was visible only when we looked at what the labels contained.

Reproducibility. The whole study is one self-contained Colab notebook: environment, expert, training, evaluation and videos, with the settings of every run at the top.

References

- [1] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, S. Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. RSS 2023; IJRR 2024. arXiv:2303.04137.
- [2] P. Florence et al. Implicit Behavioral Cloning. CoRL 2021. arXiv:2109.00137.
- [3] T. Z. Zhao, V. Kumar, S. Levine, C. Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (ACT/ALOHA). RSS 2023. arXiv:2304.13705.
- [4] S. Ross, G. Gordon, J. A. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. AISTATS 2011.
- [5] J. Grannen, Y. Wu, B. Vu, D. Sadigh. Stabilize to Act: Learning to Coordinate for Bimanual Manipulation. CoRL 2023. arXiv:2309.01087.
- [6] D. Dong, M. Bhatt, S. Choi, N. Mehr. MIMIC-D: Multi-modal Imitation for Multi-agent Coordination with Decentralized Diffusion Policies. 2025. arXiv:2509.14159.
- [7] A. Palma et al. Bimanual Robot Manipulation via Multi-Agent In-Context Learning. 2026. arXiv:2604.20348.
- [8] L. Peters, L. Ferranti, A. Bajcsy, J. Alonso-Mora. Generating Multi-Agent Behavior Without Multi-Agent Demonstrations. 2026. arXiv:2605.11485.
- [9] A. Lee, I. Chuang, L.-Y. Chen, I. Soltani. InterACT: Interdependency Aware Action Chunking with Hierarchical Attention Transformers for Bimanual Manipulation. CoRL 2024. arXiv:2409.07914.
- [10] T. Motoda et al. Learning Bimanual Manipulation via Action Chunking and Inter-Arm Coordination with Transformers. 2025. arXiv:2503.13916.
- [11] G. Lu et al. AnyBimanual: Transferring Unimanual Policy for General Bimanual Manipulation. ICCV 2025. arXiv:2412.06779.
- [12] J. Im et al. TwinVLA: Data-Efficient Bimanual Manipulation with Twin Single-Arm Vision-Language-Action Models. ICLR 2026. arXiv:2511.05275.
- [13] S. Liu et al. RDT-1B: a Diffusion Foundation Model for Bimanual Manipulation. ICLR 2025. arXiv:2410.07864.
- [14] K. Black et al. π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. arXiv:2410.24164.
- [15] N. Gkanatsios et al. 3D FlowMatch Actor: Unified 3D Policy for Single- and Dual-Arm Manipulation. 2025. arXiv:2508.11002.
- [16] Z. Zhu et al. MADiff: Offline Multi-agent Learning with Diffusion Models. NeurIPS 2024. arXiv:2305.17330.
- [17] M. Wen et al. Multi-Agent Reinforcement Learning is a Sequence Modeling Problem. NeurIPS 2022. arXiv:2205.14953.
- [18] L. Rowe et al. FJMP: Factorized Joint Multi-Agent Motion Prediction. CVPR 2023. arXiv:2211.16197.
- [19] X. Li et al. GLIDE: Planning-Guided Diffusion Policy Learning for Generalizable Contact-Rich Bimanual Manipulation. 2024. arXiv:2412.02676.
- [20] Y. Peng et al. BiCoord: A Bimanual Manipulation Benchmark towards Long-Horizon Spatial-Temporal Coordination. 2026. arXiv:2604.05831.
- [21] E. Chisari et al. Learning Robotic Manipulation Policies from Point Clouds with Conditional Flow Matching. CoRL 2024. arXiv:2409.07343.
- [22] Q. Zhang et al. FlowPolicy: Enabling Fast and Robust 3D Flow-based Policy via Consistency Flow Matching. AAAI 2025. arXiv:2412.04987.
- [23] S. Lee, Y. Kang, H. Kuo. Diff-DAgger: Uncertainty Estimation with Diffusion Policy for Robotic Manipulation. ICRA 2025. arXiv:2410.14868.
- [24] D. Wang et al. Equivariant Diffusion Policy. CoRL 2024. arXiv:2407.01812.
- [25] Y. Weng et al. Temporal Action Selection for Action Chunking. 2025. arXiv:2511.04421.